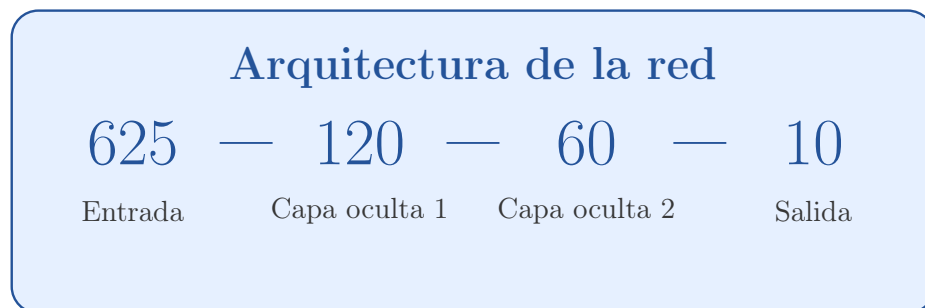


Implementación desde cero de una red neuronal

Detección de dígitos en una malla de 25×25 píxeles



Propagación hacia delante, función de coste,
retropropagación y descenso del gradiente
aplicados a una red neuronal implementada desde
cero en Python.

Mario Gabarrón Espín

Proyecto personal de Matemáticas e Informática

25 de junio de 2026

Resumen

En esta memoria se presenta el diseño e implementación conceptual de una red neuronal artificial construida desde cero en Python, sin utilizar bibliotecas especializadas de aprendizaje automático como TensorFlow o PyTorch.

El objetivo del proyecto es construir una red capaz de procesar una imagen en blanco y negro de tamaño 25×25 píxeles y producir una salida de diez componentes, donde cada componente representa la activación asociada a uno de los dígitos del 0 al 9.

La arquitectura implementada consta de cuatro capas:

$$625 \longrightarrow 120 \longrightarrow 60 \longrightarrow 10.$$

La memoria describe los fundamentos matemáticos de la red: neuronas artificiales, matrices de pesos, sesgos, función sigmoide, propagación hacia delante, función de coste, regla de la cadena, retropropagación y descenso del gradiente.

Se presta especial atención a la notación de índices, ya que la implementación utiliza la convención:

$$w_{jk}^{(\ell)} = \text{peso desde la neurona } j \text{ de la capa } \ell \text{ hasta la neurona } k \text{ de la capa } \ell + 1.$$

Esta convención se mantiene de forma coherente a lo largo de todo el documento.

1 Introducción

Una red neuronal artificial es un modelo matemático inspirado de forma muy abstracta en el funcionamiento de redes de neuronas biológicas. Desde el punto de vista matemático, una red neuronal puede entenderse como una composición de funciones dependiente de una gran cantidad de parámetros.

En este proyecto se ha construido una red neuronal desde cero con el objetivo de comprender internamente qué ocurre durante el aprendizaje. En lugar de utilizar una librería externa que oculte los detalles, se han implementado manualmente las operaciones principales:

- inicialización de la red,
- propagación hacia delante,
- cálculo de la función de coste,
- retropropagación,
- cálculo del gradiente,
- actualización de pesos mediante descenso del gradiente.

El proyecto no pretende competir en rendimiento con bibliotecas profesionales, sino servir como ejercicio matemático y computacional para entender los mecanismos internos del aprendizaje supervisado.

1.1 Objetivo general

El objetivo general es construir una red neuronal capaz de recibir una matriz de activaciones de tamaño 25×25 y producir una predicción entre diez posibles clases.

Cada píxel de la imagen se interpreta como un número real entre 0 y 1:

$$0 = \text{píxel apagado}, \quad 1 = \text{píxel encendido}.$$

También podrían admitirse valores intermedios para representar tonos de gris.

1.2 Objetivos específicos

Los objetivos específicos son:

1. Representar una imagen 25×25 como un vector de dimensión 625.
2. Construir una red neuronal con cuatro capas.

3. Implementar la función sigmoide como función de activación.
4. Calcular las activaciones capa a capa mediante propagación hacia delante.
5. Definir una función de coste que mida el error de la predicción.
6. Derivar la retropropagación usando la regla de la cadena.
7. Calcular el gradiente de la función de coste respecto a todos los pesos.
8. Actualizar los pesos mediante descenso del gradiente.

2 Representación de la entrada

La entrada original del problema es una matriz de tamaño 25×25 :

$$X = \begin{pmatrix} x_{00} & x_{01} & \cdots & x_{0,24} \\ x_{10} & x_{11} & \cdots & x_{1,24} \\ \vdots & \vdots & \ddots & \vdots \\ x_{24,0} & x_{24,1} & \cdots & x_{24,24} \end{pmatrix}, \quad x_{ij} \in [0, 1].$$

Cada entrada x_{ij} representa el nivel de activación de un píxel.

Para introducir esta imagen en una red neuronal completamente conectada, se transforma la matriz en un vector de dimensión 625:

$$X \in [0, 1]^{25 \times 25} \quad \mapsto \quad a^{(0)} \in [0, 1]^{625}.$$

En la implementación se recorre la matriz por filas. Es decir,

$$a^{(0)} = (x_{00}, x_{01}, \dots, x_{0,24}, x_{10}, x_{11}, \dots, x_{24,24}).$$

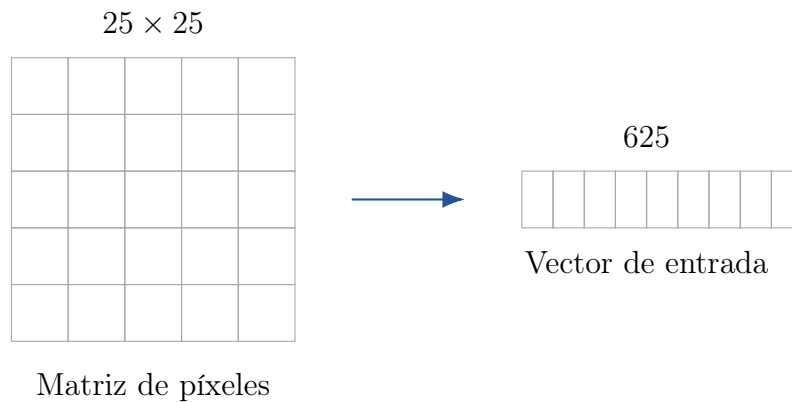


Figura 1: Conversión de una malla 25×25 en un vector de dimensión 625.

3 Arquitectura de la red neuronal

La red implementada tiene cuatro capas:

$$n_0 = 625, \quad n_1 = 120, \quad n_2 = 60, \quad n_3 = 10.$$

La capa 0 es la capa de entrada. Las capas 1 y 2 son capas ocultas. La capa 3 es la capa de salida.

$$625 \longrightarrow 120 \longrightarrow 60 \longrightarrow 10.$$

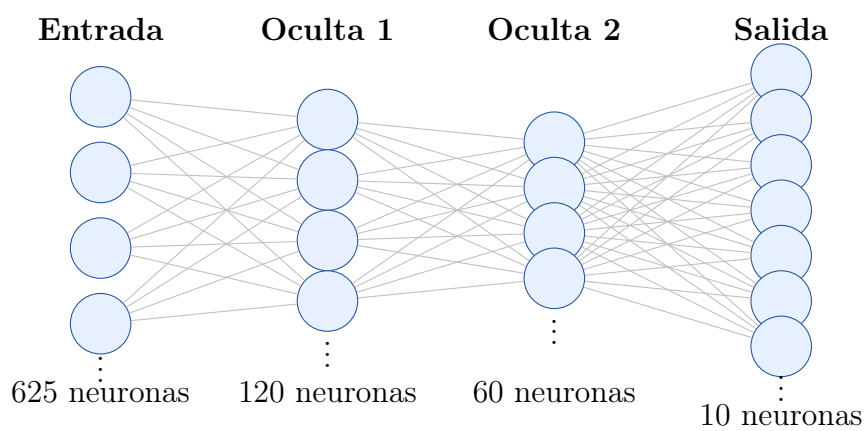


Figura 2: Esquema simplificado de la arquitectura $625 \rightarrow 120 \rightarrow 60 \rightarrow 10$.

3.1 Matrices de pesos

Entre cada par de capas consecutivas hay una matriz de pesos.

Como la red tiene cuatro capas, existen tres matrices de pesos:

$$W^{(0)} \in \mathbb{R}^{625 \times 120}, \quad W^{(1)} \in \mathbb{R}^{120 \times 60}, \quad W^{(2)} \in \mathbb{R}^{60 \times 10}.$$

La convención utilizada es:

$$w_{jk}^{(\ell)} = \text{peso desde la neurona } j \text{ de la capa } \ell \text{ hasta la neurona } k \text{ de la capa } \ell + 1.$$

Por tanto, el primer índice indica la neurona de origen y el segundo índice indica la neurona de destino.

Matriz	Conexión	Dimensión	Interpretación
$W^{(0)}$	capa 0 $\rightarrow$ 1	625×120	entrada $\rightarrow$ oculta 1
$W^{(1)}$	capa 1 $\rightarrow$ 2	120×60	oculta 1 $\rightarrow$ oculta 2
$W^{(2)}$	capa 2 $\rightarrow$ 3	60×10	oculta 2 $\rightarrow$ salida

Tabla 1: Matrices de pesos de la red neuronal.

4 Modelo de neurona artificial

Cada neurona almacena principalmente tres cantidades:

- su índice dentro de la capa,
- su activación,
- su sesgo,
- su preactivación z .

Para una neurona k de la capa $\ell + 1$, se define primero la preactivación:

$$z_k^{(\ell+1)} = \sum_{j=0}^{n_\ell-1} a_j^{(\ell)} w_{jk}^{(\ell)} + b_k^{(\ell+1)}.$$

Después se aplica una función de activación:

$$a_k^{(\ell+1)} = \sigma \left(z_k^{(\ell+1)} \right).$$

En este proyecto se utiliza la función sigmoide.

4.1 Función sigmoide

La función sigmoide se define como

$$\sigma(x) = \frac{1}{1 + e^{-x}}.$$

Su imagen está contenida en el intervalo $(0, 1)$, lo que resulta útil para interpretar las activaciones como valores de intensidad o de confianza.

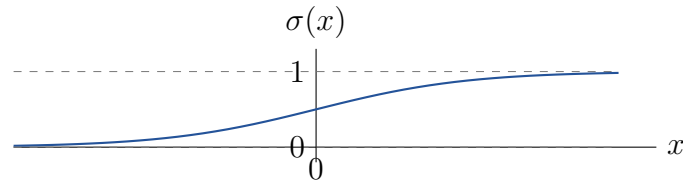


Figura 3: Función sigmoide $\sigma(x) = \frac{1}{1+e^{-x}}$.

La derivada de la sigmoide es

$$\sigma'(x) = \frac{e^{-x}}{(1 + e^{-x})^2}.$$

Además, usando que

$$\sigma(x) = \frac{1}{1 + e^{-x}},$$

se obtiene la identidad

$$\boxed{\sigma'(x) = \sigma(x)(1 - \sigma(x)).}$$

Esta forma es especialmente útil en la implementación porque durante la propagación hacia delante ya se ha calculado la activación $a = \sigma(z)$.

5 Propagación hacia delante

La propagación hacia delante consiste en calcular las activaciones de todas las capas a partir de una entrada.

Partiendo del vector de entrada

$$a^{(0)} \in \mathbb{R}^{625},$$

se calculan sucesivamente

$$a^{(1)}, \quad a^{(2)}, \quad a^{(3)}.$$

Para cada capa se usa la fórmula:

$$z_k^{(\ell+1)} = \sum_{j=0}^{n_\ell-1} a_j^{(\ell)} w_{jk}^{(\ell)} + b_k^{(\ell+1)},$$

$$a_k^{(\ell+1)} = \sigma(z_k^{(\ell+1)}).$$

5.1 Forma matricial

Si se agrupan las activaciones en vectores fila, la propagación puede escribirse como

$$z^{(\ell+1)} = a^{(\ell)} W^{(\ell)} + b^{(\ell+1)},$$

$$a^{(\ell+1)} = \sigma(z^{(\ell+1)}),$$

donde la sigmoide se aplica componente a componente.

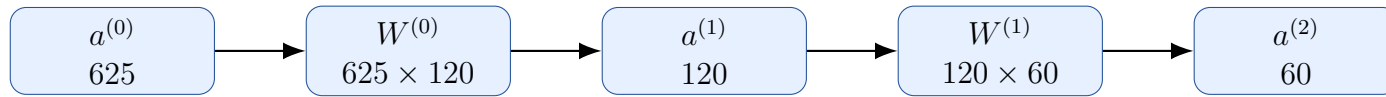


Figura 4: Flujo de la propagación hacia delante.

6 Salida de la red y vector objetivo

La salida de la red es un vector de dimensión 10:

$$a^{(3)} = (a_0^{(3)}, a_1^{(3)}, \dots, a_9^{(3)}).$$

Cada componente representa la activación asociada a un dígito.

Por ejemplo:

$$a_0^{(3)} \approx \text{confianza de que el número sea 0,}$$

$$a_7^{(3)} \approx \text{confianza de que el número sea 7.}$$

El vector objetivo se representa mediante codificación *one-hot*. Por ejemplo, si la imagen representa un 3, el vector objetivo es

$$y = (0, 0, 0, 1, 0, 0, 0, 0, 0, 0).$$

En general:

$$y_k = \begin{cases} 1, & \text{si la imagen pertenece a la clase } k, \\ 0, & \text{en caso contrario.} \end{cases}$$

7 Función de coste

Para medir el error de la red se utiliza una función de coste cuadrática media sobre las diez neuronas de salida:

$$C = \frac{1}{10} \sum_{k=0}^9 \left(a_k^{(3)} - y_k \right)^2.$$

Esta función compara la salida de la red $a^{(3)}$ con el vector objetivo y .

7.1 Interpretación

Si la red acierta perfectamente, entonces

$$a^{(3)} = y,$$

y por tanto

$$C = 0.$$

Si la red se equivoca, las diferencias

$$a_k^{(3)} - y_k$$

serán no nulas y el coste será positivo.

7.2 Derivada respecto a la activación final

Para aplicar retropropagación, se necesita derivar C respecto a las activaciones de salida.

Dado que

$$C = \frac{1}{10} \sum_{r=0}^9 (a_r^{(3)} - y_r)^2,$$

derivamos respecto a una componente concreta $a_k^{(3)}$:

$$\frac{\partial C}{\partial a_k^{(3)}} = \frac{1}{10} \cdot 2(a_k^{(3)} - y_k).$$

Por tanto,

$$\frac{\partial C}{\partial a_k^{(3)}} = \frac{1}{5}(a_k^{(3)} - y_k).$$

El factor $\frac{1}{5}$ aparece porque se ha dividido el coste entre 10 y después se ha derivado el cuadrado.

8 Descenso del gradiente

La función de coste depende de los pesos de la red. Aunque inicialmente se escribe como una función de las activaciones finales,

$$C = C(a^{(3)}, y),$$

las activaciones finales dependen de todos los pesos:

$$a^{(3)} = a^{(3)}(W^{(0)}, W^{(1)}, W^{(2)}).$$

Por tanto, desde el punto de vista del entrenamiento, se optimiza una función

$$C = C(W^{(0)}, W^{(1)}, W^{(2)}).$$

El gradiente de C indica la dirección de máximo crecimiento. Para disminuir el coste se avanza en la dirección opuesta:

$$W^{(\ell)} \leftarrow W^{(\ell)} - \eta \frac{\partial C}{\partial W^{(\ell)}}.$$

El parámetro $\eta > 0$ se denomina tasa de aprendizaje o *learning rate*. Controla el tamaño del paso.

Si η es demasiado pequeño, el entrenamiento será muy lento. Si es demasiado grande, el algoritmo puede no converger.

9 Retropropagación

La retropropagación es el algoritmo que permite calcular eficientemente el gradiente de la función de coste respecto a todos los pesos de la red.

La idea central es aplicar repetidamente la regla de la cadena desde la capa de salida hacia las capas anteriores.

9.1 Gradiente de la última matriz de pesos

Consideremos primero un peso de la última matriz:

$$w_{jk}^{(2)},$$

que conecta la neurona j de la capa 2 con la neurona k de la capa 3.

Según la convención del proyecto:

$$j = 0, \dots, 59, \quad k = 0, \dots, 9.$$

Se quiere calcular

$$\frac{\partial C}{\partial w_{jk}^{(2)}}.$$

Como

$$z_k^{(3)} = \sum_{j=0}^{59} a_j^{(2)} w_{jk}^{(2)} + b_k^{(3)},$$

se aplica la regla de la cadena:

$$\frac{\partial C}{\partial w_{jk}^{(2)}} = \frac{\partial C}{\partial a_k^{(3)}} \frac{\partial a_k^{(3)}}{\partial z_k^{(3)}} \frac{\partial z_k^{(3)}}{\partial w_{jk}^{(2)}}.$$

Ya se ha calculado que

$$\frac{\partial C}{\partial a_k^{(3)}} = \frac{1}{5} (a_k^{(3)} - y_k).$$

Además,

$$\frac{\partial a_k^{(3)}}{\partial z_k^{(3)}} = \sigma'(z_k^{(3)}),$$

y

$$\frac{\partial z_k^{(3)}}{\partial w_{jk}^{(2)}} = a_j^{(2)}.$$

Por tanto,

$$\frac{\partial C}{\partial w_{jk}^{(2)}} = \frac{1}{5}(a_k^{(3)} - y_k)\sigma'(z_k^{(3)})a_j^{(2)}.$$

Esta fue la primera derivación directa del gradiente.

9.2 Aparición natural del término delta

En la expresión anterior aparece repetidamente el producto

$$\frac{1}{5}(a_k^{(3)} - y_k)\sigma'(z_k^{(3)}).$$

Se define entonces

$$\delta_k^{(3)} = \frac{\partial C}{\partial z_k^{(3)}}.$$

Para la capa de salida:

$$\delta_k^{(3)} = \frac{1}{5}(a_k^{(3)} - y_k)\sigma'(z_k^{(3)}).$$

Con esta notación, la derivada respecto a un peso de la última capa queda simplemente:

$$\frac{\partial C}{\partial w_{jk}^{(2)}} = a_j^{(2)}\delta_k^{(3)}.$$

La introducción de δ no cambia la matemática. Simplemente da nombre a una derivada parcial que aparece repetidamente.

9.3 Retropropagación hacia la capa oculta 2

Ahora se quiere calcular el gradiente respecto a un peso de la matriz $W^{(1)}$:

$$w_{jk}^{(1)},$$

que conecta la neurona j de la capa 1 con la neurona k de la capa 2.

En este caso, la neurona k de la capa 2 influye en todas las neuronas de salida. Por eso aparece un sumatorio.

Aplicando la regla de la cadena:

$$\frac{\partial C}{\partial w_{jk}^{(1)}} = \sum_{r=0}^9 \frac{\partial C}{\partial a_r^{(3)}} \frac{\partial a_r^{(3)}}{\partial z_r^{(3)}} \frac{\partial z_r^{(3)}}{\partial a_k^{(2)}} \frac{\partial a_k^{(2)}}{\partial z_k^{(2)}} \frac{\partial z_k^{(2)}}{\partial w_{jk}^{(1)}}.$$

Como

$$\frac{\partial z_r^{(3)}}{\partial a_k^{(2)}} = w_{kr}^{(2)},$$

se obtiene

$$\frac{\partial C}{\partial w_{jk}^{(1)}} = \left(\sum_{r=0}^9 \delta_r^{(3)} w_{kr}^{(2)} \right) \sigma'(z_k^{(2)}) a_j^{(1)}.$$

Se define

$$\delta_k^{(2)} = \left(\sum_{r=0}^9 w_{kr}^{(2)} \delta_r^{(3)} \right) \sigma'(z_k^{(2)}).$$

Entonces

$$\frac{\partial C}{\partial w_{jk}^{(1)}} = a_j^{(1)} \delta_k^{(2)}.$$

9.4 Retropropagación hacia la capa oculta 1

De forma análoga, para la capa anterior:

$$\delta_k^{(1)} = \left(\sum_{r=0}^{59} w_{kr}^{(1)} \delta_r^{(2)} \right) \sigma'(z_k^{(1)}).$$

Y el gradiente de un peso de $W^{(0)}$ queda:

$$\frac{\partial C}{\partial w_{jk}^{(0)}} = a_j^{(0)} \delta_k^{(1)}.$$

9.5 Fórmula general

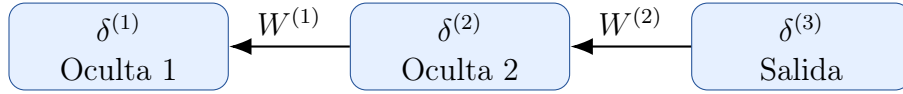
Para una capa oculta cualquiera, se tiene:

$$\delta_k^{(\ell)} = \left(\sum_{r=0}^{n_{\ell+1}-1} w_{kr}^{(\ell)} \delta_r^{(\ell+1)} \right) \sigma'(z_k^{(\ell)}).$$

Y el gradiente de los pesos queda:

$$\frac{\partial C}{\partial w_{jk}^{(\ell)}} = a_j^{(\ell)} \delta_k^{(\ell+1)}.$$

Esta es la fórmula central de la retropropagación.



El error se propaga desde la salida hacia atrás.

Figura 5: Flujo de la retropropagación mediante los términos δ .

10 Convención de índices

Uno de los aspectos más importantes del proyecto es mantener una convención de índices coherente.

La convención utilizada es:

$w_{jk}^{(\ell)}$ = peso desde la neurona j de la capa ℓ hacia la neurona k de la capa $\ell + 1$.

Por tanto:

j = índice de origen,

k = índice de destino.

La fórmula de propagación hacia delante es

$$z_k^{(\ell+1)} = \sum_{j=0}^{n_{\ell}-1} a_j^{(\ell)} w_{jk}^{(\ell)} + b_k^{(\ell+1)}.$$

Aquí el índice j se suma porque recorre todas las neuronas de la capa anterior. El índice k no se suma porque identifica la neurona cuyo potencial se está calculando.

Símbolo	Significado	Rango típico
ℓ	índice de capa	0, 1, 2, 3
j	neurona de origen	$0, \dots, n_\ell - 1$
k	neurona de destino	$0, \dots, n_{\ell+1} - 1$
r	índice auxiliar para capa siguiente	depende de la capa
$a_j^{(\ell)}$	activación de la neurona j en capa ℓ	real en $(0, 1)$
$z_k^{(\ell)}$	preactivación de la neurona k en capa ℓ	real
$w_{jk}^{(\ell)}$	peso desde j hasta k	real
$\delta_k^{(\ell)}$	$\frac{\partial C}{\partial z_k^{(\ell)}}$	real

Tabla 2: Resumen de la notación de índices.

11 Descripción de las funciones principales

En esta sección se describen las funciones principales del programa sin incluir el código fuente completo.

11.1 `initNeuronalNetwork`

Esta función recibe una matriz de tamaño 25×25 con valores en $[0, 1]$.

Su objetivo es convertir dicha matriz en un vector de longitud 625 y cargar esos valores como activaciones de la capa de entrada.

Matemáticamente, realiza la operación:

$$[0, 1]^{25 \times 25} \longrightarrow [0, 1]^{625}.$$

No modifica pesos ni sesgos. Solo actualiza las activaciones iniciales.

11.2 `parseMatrixToVectorRn2`

Esta función transforma una matriz cuadrada en un vector recorriéndola por filas.

Su papel es puramente auxiliar, pero es importante porque define cómo se corresponde cada píxel con una neurona de entrada.

Si se cambia el orden de recorrido, también cambiaría la interpretación espacial de la imagen.

11.3 `forwarding`

Esta función implementa la propagación hacia delante.

Para cada capa posterior a la entrada, calcula:

$$z_k^{(\ell+1)} = \sum_j a_j^{(\ell)} w_{jk}^{(\ell)} + b_k^{(\ell+1)},$$

y después

$$a_k^{(\ell+1)} = \sigma(z_k^{(\ell+1)}).$$

La función devuelve la capa final, formada por diez neuronas.

11.4 coste

Esta función recibe la salida de la red y el vector objetivo.

Calcula:

$$C = \frac{1}{10} \sum_{k=0}^9 (a_k^{(3)} - y_k)^2.$$

Devuelve un único número real que mide el error de la predicción.

11.5 retropropagacion

Esta función calcula el gradiente de la función de coste respecto a todas las matrices de pesos.

Devuelve una lista con las matrices de gradientes de los pesos y los términos delta necesarios para actualizar los sesgos:

$$[dW^{(0)}, dW^{(1)}, dW^{(2)}, \delta^{(1)}, \delta^{(2)}, \delta^{(3)}].$$

Cada matriz de gradientes tiene el mismo tamaño que su matriz de pesos correspondiente, y cada vector delta tiene la dimensión de la capa asociada.

11.6 entrenar

La función de entrenamiento aplica repetidamente el ciclo fundamental:

entrada $\longrightarrow$ forward $\longrightarrow$ coste $\longrightarrow$ retropropagación $\longrightarrow$ actualización de pesos.

Para cada ejemplo de entrenamiento se calcula el gradiente y se actualizan los pesos en la dirección de descenso.

11.7 predecir

La función de predicción toma una imagen, ejecuta la propagación hacia delante y selecciona la neurona de salida con mayor activación:

$$\hat{k} = \arg \max_k a_k^{(3)}.$$

El índice $\hat{k}$ se interpreta como el dígito predicho.

12 Algoritmo general de entrenamiento

El entrenamiento completo de la red puede resumirse en los siguientes pasos:

1. Inicializar pesos y sesgos.
2. Para cada época:
 - a) Para cada imagen de entrenamiento:
 - 1) Cargar la imagen en la capa de entrada.
 - 2) Calcular la salida mediante propagación hacia delante.
 - 3) Calcular el coste.
 - 4) Calcular el gradiente mediante retropropagación.
 - 5) Actualizar los pesos usando descenso del gradiente.

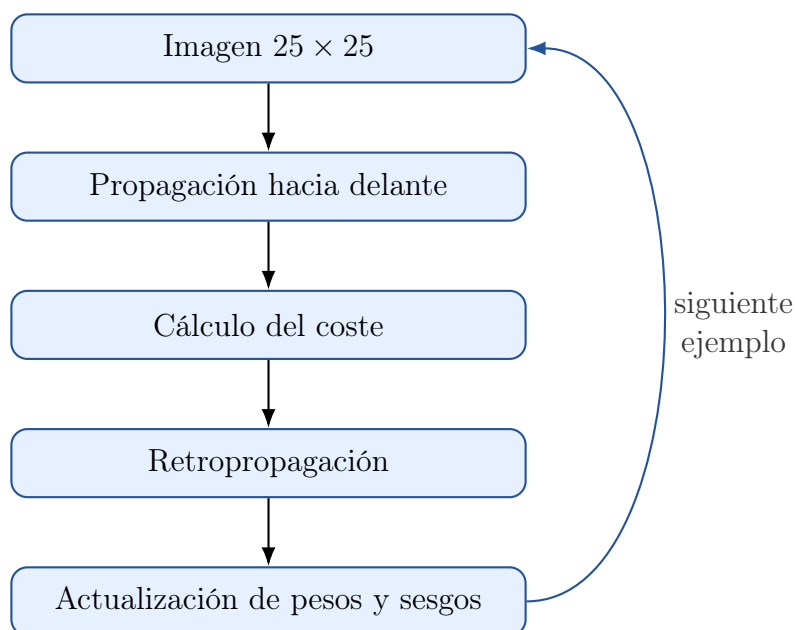


Figura 6: Ciclo completo de entrenamiento de la red neuronal.

13 Inicialización de pesos

Inicialmente los pesos se generan aleatoriamente.

Una observación importante es que si todos los pesos se toman en el intervalo $[0, 1)$, las sumas

$$z_k^{(\ell+1)} = \sum_j a_j^{(\ell)} w_{jk}^{(\ell)}$$

tienden a ser muy grandes y positivas, especialmente en capas con muchas neuronas de entrada.

Por ejemplo, en la primera capa oculta una neurona recibe 625 entradas. Si las activaciones y los pesos tienen media aproximadamente 0,5, entonces la suma media puede ser del orden de

$$625 \cdot 0,5 \cdot 0,5 = 156,25.$$

Entonces

$$\sigma(156,25) \approx 1.$$

Esto produce saturación de la sigmoide.

Por tanto, para facilitar el entrenamiento, es conveniente utilizar inicializaciones con pesos pequeños y centrados alrededor de cero. En redes neuronales modernas existen estrategias como la inicialización de Xavier o He, diseñadas precisamente para evitar que las activaciones crezcan o desaparezcan al avanzar por las capas.

14 Comprobación del gradiente

Una técnica útil para verificar la retropropagación es el *gradient checking*.

La idea consiste en comparar la derivada calculada mediante retropropagación con una aproximación numérica.

Para un peso w , se aproxima:

$$\frac{\partial C}{\partial w} \approx \frac{C(w + \varepsilon) - C(w - \varepsilon)}{2\varepsilon}.$$

Si la retropropagación está bien implementada, entonces

$$\frac{\partial C}{\partial w}$$

debe ser muy similar al valor numérico anterior para valores pequeños de ε , por ejemplo $\varepsilon = 10^{-6}$.

Esta técnica permite detectar errores de índices, factores constantes incorrectos o derivadas mal calculadas.

15 Complejidad computacional

El coste principal de la propagación hacia delante está en las multiplicaciones matriz-vector entre capas.

El número total de pesos es:

$$625 \cdot 120 + 120 \cdot 60 + 60 \cdot 10.$$

Calculando:

$$625 \cdot 120 = 75000,$$

$$120 \cdot 60 = 7200,$$

$$60 \cdot 10 = 600.$$

Por tanto:

$$\boxed{75000 + 7200 + 600 = 82800}$$

La red contiene 82 800 pesos.

Cada propagación hacia delante requiere, en esencia, recorrer todos esos pesos. La retropropagación tiene un orden de complejidad similar, ya que calcula una derivada asociada a cada peso.

16 Interfaz gráfica

Además de la red neuronal, el proyecto incluye una interfaz gráfica para dibujar números en una cuadrícula de 25×25 píxeles.

La interfaz permite:

- dibujar manualmente un número,

- borrar partes del dibujo,
- guardar ejemplos propios,
- entrenar la red con ejemplos,
- reconocer el número dibujado,
- visualizar las activaciones de las diez neuronas de salida.

La interfaz no forma parte del núcleo matemático del proyecto, pero resulta útil para probar de forma visual el comportamiento de la red.

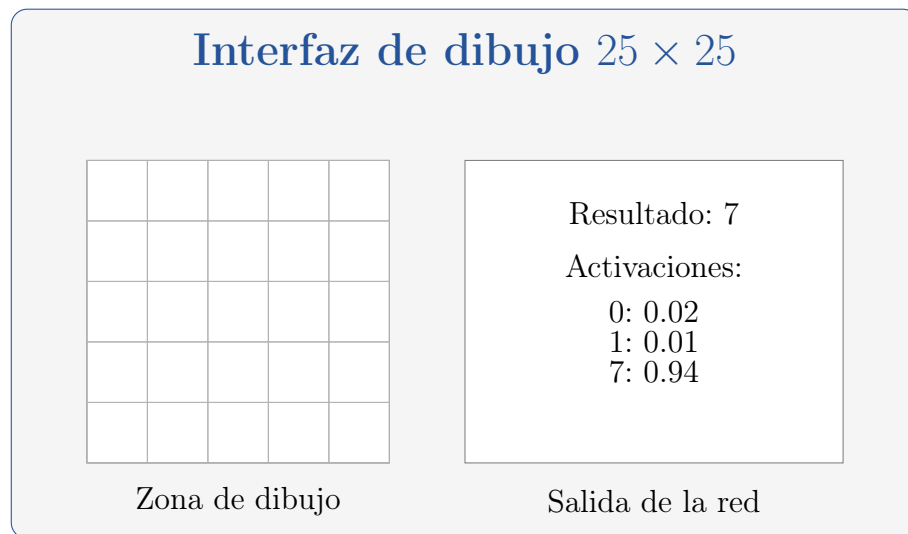


Figura 7: Esquema conceptual de la interfaz gráfica.

17 Resultados esperados

Al inicio del entrenamiento, las salidas de la red son esencialmente aleatorias, ya que los pesos se inicializan de forma aleatoria.

Conforme se repiten ciclos de entrenamiento, se espera que:

- el coste medio disminuya,
- las activaciones de salida se aproximen al vector objetivo,
- la neurona correspondiente al dígito correcto tienda a activarse más que las demás,
- la precisión sobre ejemplos de prueba aumente.

El proceso de aprendizaje consiste precisamente en modificar los pesos para reducir la función de coste.

18 Limitaciones

La red implementada es una red completamente conectada sencilla. Esto implica varias limitaciones:

- No aprovecha explícitamente la estructura espacial de las imágenes.
- Tiene muchos parámetros para una entrada relativamente pequeña.
- Puede ser sensible a traslaciones o deformaciones del dígito.
- La función sigmoide puede saturarse si los pesos no se inicializan adecuadamente.
- El entrenamiento manual puede ser más lento que en bibliotecas optimizadas.

Para reconocimiento de imágenes, las redes convolucionales suelen ser más adecuadas, ya que explotan la estructura local de los píxeles. Sin embargo, una red completamente conectada es una excelente primera implementación para comprender los fundamentos matemáticos del aprendizaje neuronal.

19 Posibles mejoras

Algunas mejoras posibles son:

1. Guardar y cargar los sesgos junto con los pesos.
2. Usar inicialización Xavier o He.
3. Sustituir la sigmoide por ReLU en las capas ocultas.
4. Utilizar una función de coste basada en entropía cruzada.
5. Implementar entrenamiento por mini-lotes.
6. Guardar y cargar pesos entrenados.
7. Añadir regularización para evitar sobreajuste.
8. Evaluar la red con un conjunto de prueba separado.
9. Visualizar gráficamente la evolución del coste.
10. Comparar la implementación propia con una versión en PyTorch o TensorFlow.

20 Conclusión

En este proyecto se ha construido desde cero una red neuronal artificial para detección de dígitos en una malla de 25×25 píxeles.

El valor principal del proyecto no está únicamente en que la red funcione, sino en haber implementado manualmente los elementos fundamentales del aprendizaje supervisado:

- representación de entradas,
- arquitectura por capas,
- propagación hacia delante,
- función de coste,
- regla de la cadena,
- retropropagación,
- descenso del gradiente.

La derivación de la retropropagación muestra que el algoritmo no es una caja negra, sino una aplicación sistemática de la regla de la cadena. La introducción del término

$$\delta_k^{(\ell)} = \frac{\partial C}{\partial z_k^{(\ell)}}$$

permite reutilizar cálculos y generalizar el algoritmo a redes con muchas capas.

El proyecto combina programación, cálculo diferencial, álgebra lineal y optimización, mostrando cómo diferentes áreas de las Matemáticas y la Informática se integran en el diseño de modelos de aprendizaje automático.

21 Código fuente principal

En esta sección se recoge el código principal desarrollado para la implementación de la red neuronal. Se incluyen únicamente los dos módulos fundamentales del proyecto: el archivo `estructura_datos.py`, donde se define la estructura básica de las neuronas, la función sigmoide y las matrices de pesos; y el archivo `red_neuronal.py`, donde se implementan la propagación hacia delante, la función de coste, la retropropagación, el descenso del gradiente, el entrenamiento y la predicción.

21.1 estructura_datos.py

Listing 1: Código del archivo estructura_datos.py

```
1  """
2  estructura_datos.py
3
4  Estructura de datos desde cero de una red neuronal de 4 capas
5  para la detección numérica en una malla de 25x25 píxeles :)
6
7  Autor: Mario Gabarrón Espín
8  """
9  import numpy as np
10
11  NUMERO_CAPAS = 4
12
13  # Defino la neurona con su índice entre las n_L que hay en la capa L, la capa
14  # L donde está y su sesgo.
15
16  class Neurona:
17
18      def __init__(self, indice , sesgo, z):
19          self.indice = indice
20          # self.capa = capa (ya lo guardará la matriz)
21          self.activacion = 0 # CORRECCIÓN de chati, mejor
22          # ponemos a 0, no hace falta asignar aleatoriedad al estado inicial
23          self.sesgo = sesgo
24          self.z = z
25
26      #def metodo(self):
27      #    print(self.atributo1)
28
29  # Defino las matrices de pesos (habrá 3, para cada zona entre capas) de forma
30  # aleatoria
31  # Después en la parte de aprendizaje estos pesos irán cambiando con el
32  # descenso del gradiente :)
33
34  W1 = np.random.randn(625,120)*0.05
35  W2 = np.random.randn(120,60)*0.05
36  W3 = np.random.randn(60,10)*0.05
37
38  matricesPesos = [W1,W2,W3]
39
40  # Defino la función sigmoide, que tomaremos la clásica (definida de R a
41  # [0,1])
42  # (Hay muchas más eficientes pero uso la más famosa)
43
44  def sigmoide(x):
45      return 1/(1+np.exp(-x))
```

21.2 red_neuronal.py

Listing 2: Código del archivo red_neuronal.py

```
1  """
2  red_neuronal.py
3
4  Implementación desde cero de una red neuronal de 4 capas
5  para la detección numérica en una malla de 25x25 píxeles :)
6
7  Autor: Mario Gabarrón Espín
8  """
9  import numpy as np
10 from estructura_datos import Neurona, sigmoide, NUMERO_CAPAS, matricesPesos
11
12 TAMAÑOS_CAPAS = [625,120,60,10]
13
14 # Creamos las neuronas de nuestra red.
15
16 capas = []
17
18 for l in range(0, NUMERO_CAPAS):
19
20     neuronas = []
21
22     for k in range(0,TAMAÑOS_CAPAS[l]):
23         neuronas.append(Neurona(k,0,0))
24
25     capas.append(neuronas)
26
27 # Inicializar la entrada de la red neuronal dada una matriz de valores entre
28   0 y 1 en R.
29 # Entrada: matriz de valores reales en [0,1]
30
31 def initNeuronalNetwork(activacionesCapa0):
32     vectorCapa0 = parseMatrixToVectorRn2(activacionesCapa0)
33     for i in range(0,TAMAÑOS_CAPAS[0]):
34         capas[0][i].activacion = vectorCapa0[i]
35
36 # Definamos un forwarding: calcular la activación de cada una de las neuronas
37   .
38 # Entrada: 625 valores entre 0 y 1
39 # Salida: 10 neuronas
40
41 def forwarding():
42
43     for l in range(1, NUMERO_CAPAS):
44         for j in range(0, TAMAÑOS_CAPAS[l]):
45             y = 0
46             for m in range(0, TAMAÑOS_CAPAS[l-1]):
```



```

46         y += capas[l-1][m].activacion*matricesPesos[l-1][m][j] # aquí
        m recorre todas las neuronas de la capa L-1, j recorre las neuronas de
        la capa L.
47         z = y + capas[l][j].sesgo
48         capas[l][j].z = z
49         capas[l][j].activacion = sigmoide(z)
50
51     return capas[NUMERO_CAPAS-1]
52
53 def parseMatrixToVectorRn2(matrix):
54     vector = []
55     for i in range(0,len(matrix)):
56         for j in range(0, len(matrix)):
57             vector.append(matrix[i][j])
58     return vector
59
60 # Defino la función de costes.
61 # Entrada: vector de neuronas de la capa final y vector objetivo (e.g.:
        [0,0,0,1,0,0,0,0,0] representa el 3)
62
63 def coste(neuronasFinal, vectorObjetivo):
64     y=0
65     for i in range(0,TAMAÑOS_CAPAS[3]):
66         y += np.pow(neuronasFinal[i].activacion - vectorObjetivo[i],2)
67     return y/TAMAÑOS_CAPAS[3]
68
69 # Retropropagación: nos servirá para calcular el gradiente de la función C (
        coste)
70 # para así poder aplicar el método del descenso del gradiente, y hallar los m
        ínimos locales de la función de costes,
71 # para conseguir que nuestra red neuronal vaya aprendiendo.
72
73 def retropropagacion(vectorObjetivo):
74
75     dW0 = np.zeros((625,120))
76     dW1 = np.zeros((120,60))
77     dW2 = np.zeros((60,10))
78
79     delta3 = np.zeros(10)
80     delta2 = np.zeros(60)
81     delta1 = np.zeros(120)
82
83     # ===== CAPA DE SALIDA =====
84
85     for k in range(10):
86
87         a = capas[3][k].activacion
88
89         delta3[k] = ((a-vectorObjetivo[k])/5) * a * (1-a)
90

```

```
91     for j in range(60):
92         for k in range(10):
93
94             dW2[j][k] = capas[2][j].activacion * delta3[k]
95
96     # ===== CAPA OCULTA 2 =====
97
98     for k in range(60):
99
100         suma = 0
101
102         for r in range(10):
103             suma += matricesPesos[2][k][r] * delta3[r]
104
105         a = capas[2][k].activacion
106
107         delta2[k] = suma * a * (1-a)
108
109     for j in range(120):
110         for k in range(60):
111
112             dW1[j][k] = capas[1][j].activacion * delta2[k]
113
114     # ===== CAPA OCULTA 1 =====
115
116     for k in range(120):
117
118         suma = 0
119
120         for r in range(60):
121             suma += matricesPesos[1][k][r] * delta2[r]
122
123         a = capas[1][k].activacion
124
125         delta1[k] = suma * a * (1-a)
126
127     for j in range(625):
128         for k in range(120):
129
130             dW0[j][k] = capas[0][j].activacion * delta1[k]
131
132     return [dW0, dW1, dW2, delta1, delta2, delta3]
133
134 # Realizamos el descenso del gradiente modificando los pesos y los sesgos seg
    ún el gradiente (lo obtenemos de la retropropagación)
135
136 def descensoGradiente(gradiente):
137     eta = 1
138
139     matricesPesos[0] -= eta * gradiente[0]
```

```
140     matricesPesos[1] -= eta * gradiente[1]
141     matricesPesos[2] -= eta * gradiente[2]
142
143     for i in range(120):
144         capas[1][i].sesgo -= eta * gradiente[3][i]
145
146     for i in range(60):
147         capas[2][i].sesgo -= eta * gradiente[4][i]
148
149     for i in range(10):
150         capas[3][i].sesgo -= eta * gradiente[5][i]
151
152 # Entrenamos nuestro modelo
153
154 def entrenar(imagenes, etiquetas, epochs):
155     for e in range(epochs):
156
157         coste_total = 0
158
159         indices = np.random.permutation(len(imagenes))
160
161         for i in indices:
162             initNeuronalNetwork(imagenes[i])
163             salida = forwarding()
164
165             coste_total += coste(salida, etiquetas[i])
166
167             gradiente = retropropagacion(etiquetas[i])
168             descensoGradiente(gradiente)
169
170         precision = evaluar(imagenes, etiquetas)
171
172         print("Epoch", e)
173         print("Coste medio:", coste_total / len(imagenes))
174         print("Precisión:", precision)
175         print("-----")
176
177         if precision == 1.0:
178             print("Precisión perfecta. Paramos entrenamiento.")
179             break
180
181     return capas[NUMERO_CAPAS-1]
182
183 def predecir(imagen):
184
185     initNeuronalNetwork(imagen)
186     salida = forwarding()
187     activaciones = [n.activacion for n in salida]
188     return np.argmax(activaciones)
189
```

```
190
191 def evaluar(imagenes, etiquetas):
192
193     aciertos = 0
194
195     for i in range(len(imagenes)):
196         prediccion = predecir(imagenes[i])
197         real = np.argmax(etiquetas[i])
198         if prediccion == real:
199             aciertos += 1
200
201     precision = aciertos / len(imagenes)
202     print("Aciertos:", aciertos, "/", len(imagenes))
203     print("Precisión:", precision)
204
205     return precision
```